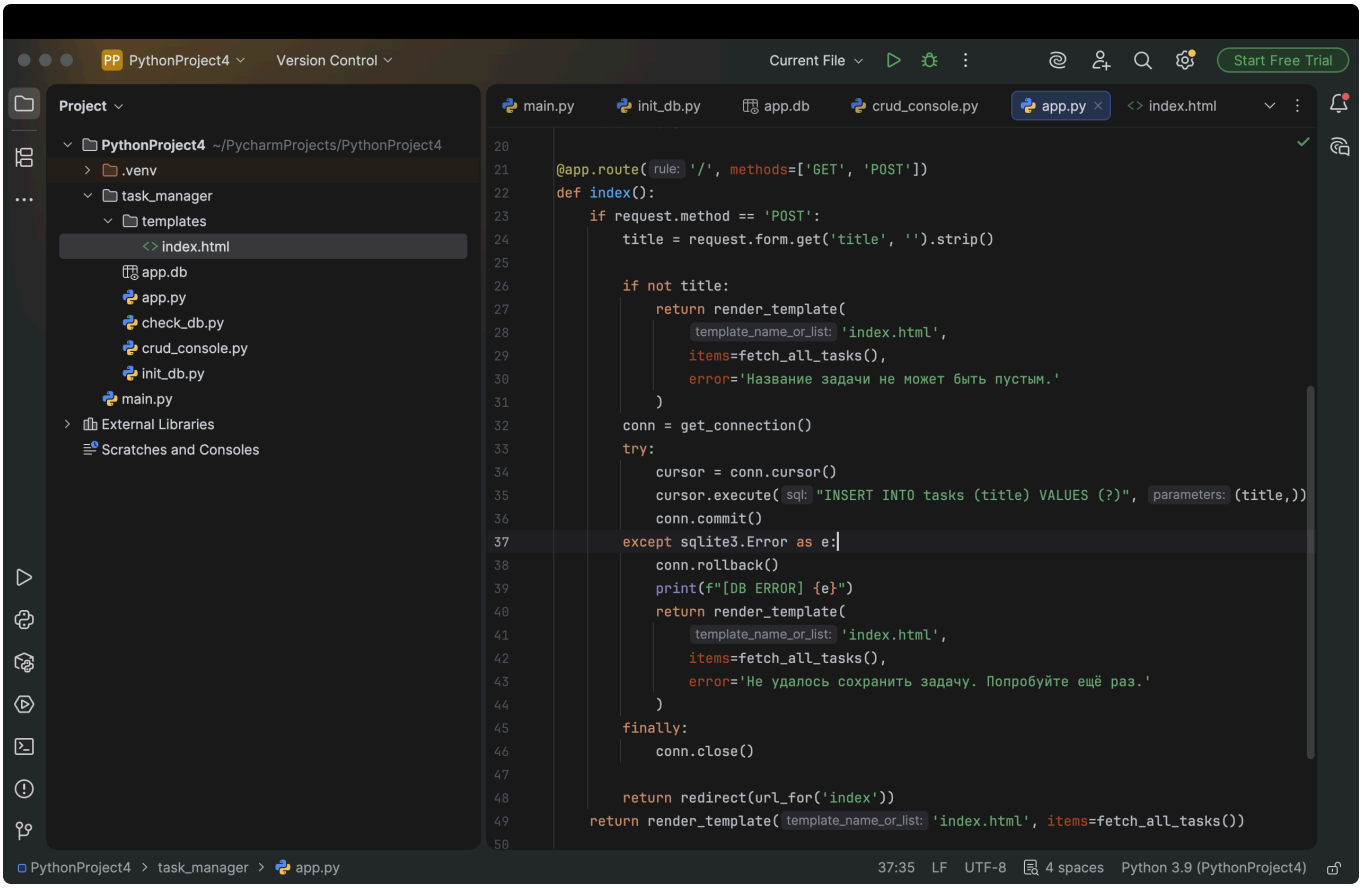
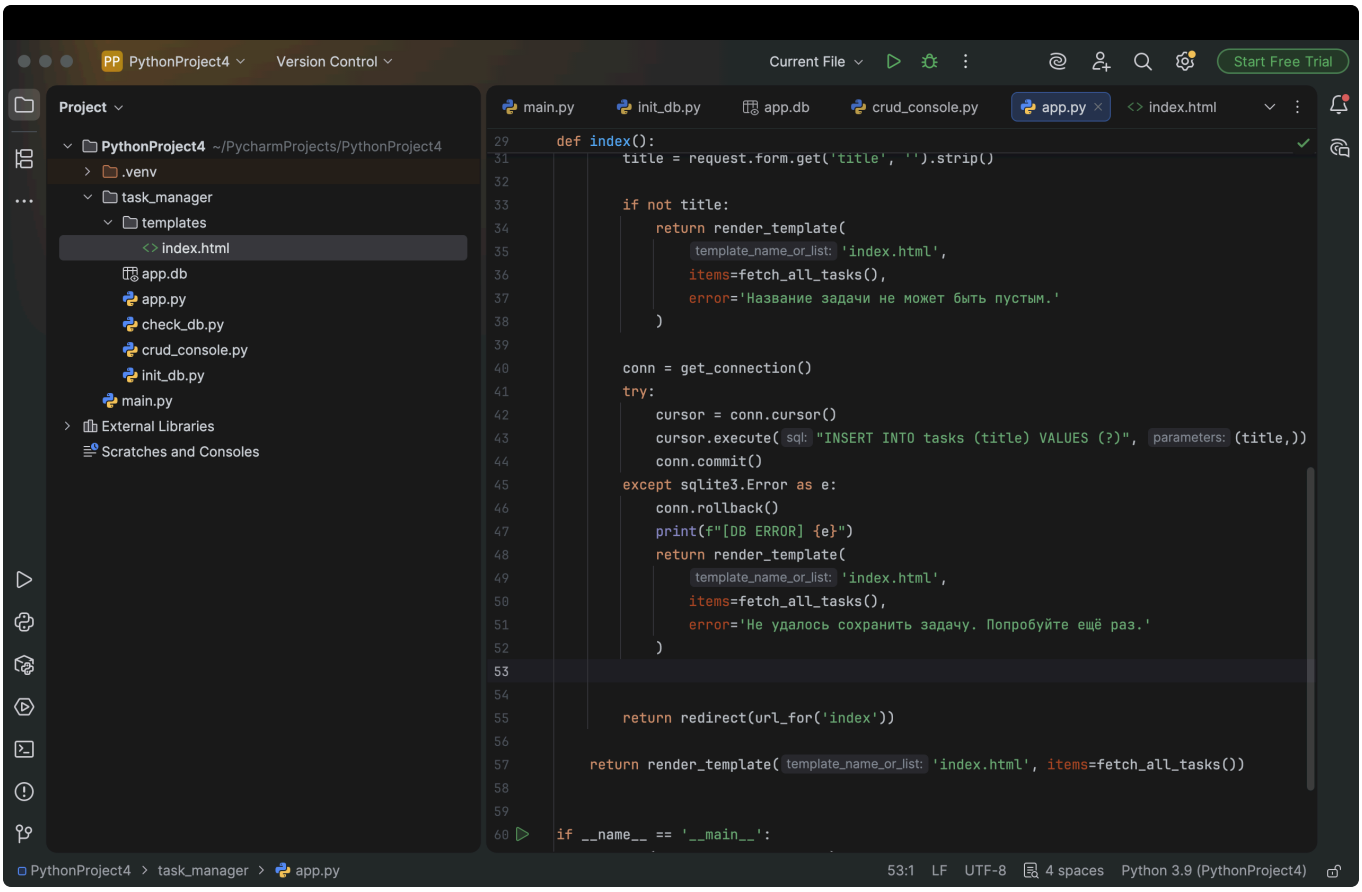
Обработка ошибок и защита соединений

Ястребкова Лилия ЗИСП11-44

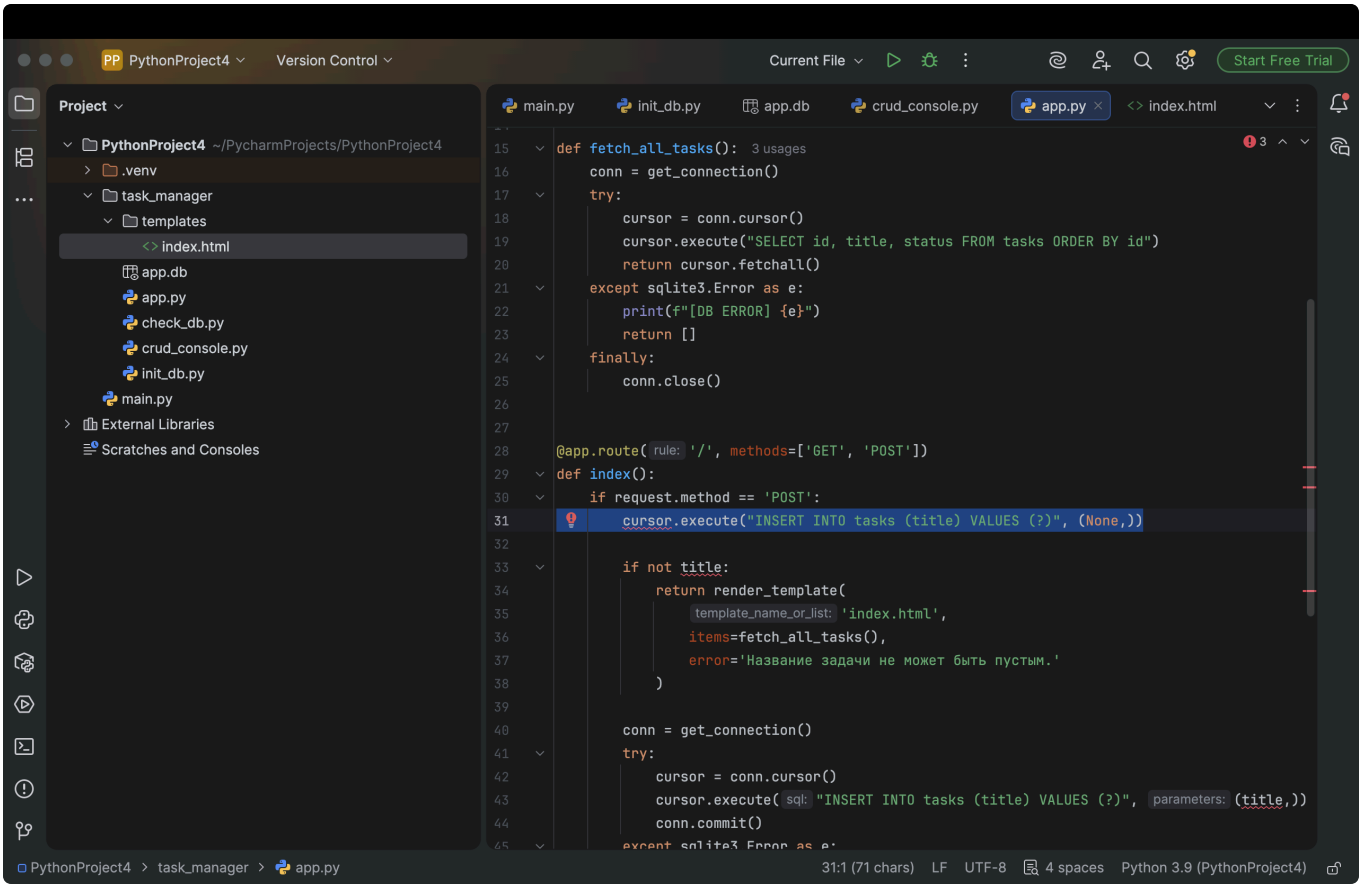
Переписываем ветку POST с try/except/finally



Шаг 3. Применяем тот же паттерн к чтению



Шаг 5. Эксперимент 1 — специально вызываем ошибку NOT NULL



1. Почему `conn.close()` нельзя оставлять просто в конце функции, а нужно помещать в `finally`?

Если `close()` стоит в конце функции после всех действий, то при возникновении исключения (например, ошибка в `execute` или `commit`) выполнение функции прервётся раньше этой строки - соединение так и не закроется. Это приведёт к тому, что файл базы останется заблокированным, другие процессы (DB Browser, консольный клиент) не смогут к ней обратиться, а в веб-приложении соединения будут «утекать» при каждой ошибке. Блок `finally` гарантирует, что `conn.close()` выполнится при любом исходе: и при успехе, и при ошибке, и даже если в середине блока был `return`.

---

2. Что делает `conn.rollback()` и в каком случае он вызывается?

`conn.rollback()` отменяет все изменения, сделанные в рамках текущей транзакции, возвращая базу в состояние, которое было до начала операции. Он вызывается в блоке `except` при возникновении ошибки (например, `sqlite3.IntegrityError` из-за нарушения `NOT NULL`). Цель - не допустить появления в базе «полузаписанных» или некорректных данных: если вставка не завершилась успешно, она полностью откатывается. Без `rollback` база может остаться в несогласованном состоянии, особенно при работе с несколькими связанными операциями.

---

3. Почему `conn` нужно открывать до `try`, а не внутри?

Переменная `conn` должна быть доступна в блоке `finally`, чтобы можно было гарантированно закрыть соединение. Если объявить `conn = get_connection()` внутри `try`, то при ошибке до этой строки переменная `conn` вообще не будет создана, и обращение к ней в `finally` вызовет `UnboundLocalError`. Открывая соединение до `try`, мы гарантируем, что если оно было успешно создано, то в `finally` оно точно будет доступно для закрытия.

---

4. Чем `except sqlite3.Error` лучше, чем `except:` без указания типа?

`except sqlite3.Error` ловит только ошибки, связанные с работой базы данных (нарушения ограничений, проблемы с подключением, ошибки SQL и т. п.), оставляя другие исключения (например, `KeyboardInterrupt` при нажатии Ctrl+C или системные ошибки) необработанными - так их можно корректно обработать на более высоком уровне или завершить работу сервера. Голый `except:` перехватывает абсолютно всё, включая сигналы прерывания и критические ошибки интерпретатора, что может скрыть реальные проблемы и помешать корректному завершению работы. Кроме того,

конкретная обработка ошибок позволяет давать пользователю более точные сообщения и вести логирование по типам ошибок.

---

5. Что произойдёт с данными, если `execute` прошёл, а `commit` упал?

Изменения, выполненные в `execute`, находятся в состоянии «в черновике» и ещё не зафиксированы на диске. Если на этапе `commit` произойдёт ошибка и не будет вызван `rollback`, транзакция останется «висящей»: данные не будут окончательно записаны, но и явно не отменены. Поведение зависит от драйвера и ОС: SQLite может откатить транзакцию при следующем открытии, но полагаться на это нельзя. В правильно написанном коде в блоке `except` должен быть `rollback()`, который явно отменит все изменения этой транзакции - база вернётся в исходное состояние.

---

6. Зачем в веб-приложении возвращать пользователю понятное сообщение вместо стектрейса?

Стек-трейс (traceback) содержит внутреннюю информацию о коде, путях к файлам и структуре приложения - это потенциальная угроза безопасности: злоумышленник может использовать эти данные для анализа уязвимостей. Кроме того, стектрейс непонятен обычному пользователю и выглядит как «страшная ошибка», что ухудшает UX. Вместо этого приложение должно перехватывать ошибки, логировать их (для разработчика) и показывать пользователю простое, понятное сообщение (например, «Не удалось сохранить задачу. Попробуйте ещё раз»), сохраняя работоспособность сервиса.

---

7. Как проверить, что `rollback` действительно откатил транзакцию?

Самый наглядный способ - искусственно вызвать ошибку после `execute`, но до `commit`. Например, временно подставить `None` в `INSERT` (что нарушит `NOT NULL`) или добавить строку `1 / 0` после `execute`. При этом:

- в блоке `except` сработает `rollback()`;
- пользователь увидит сообщение об ошибке, а не стектрейс;
- в базе не появится «мусорная» строка - количество задач останется прежним;
- соединение корректно закроется в `finally`, и другие инструменты (DB Browser, `crud_console.py`) смогут открыть базу без ошибки `database is locked`.
  Это доказывает, что транзакция была отменена, а приложение осталось устойчивым.